# Knowledge Distillation and Deployment of a Command-Parsing LM on a Local Cloud

Cloud Computing (CS3081) mini project. **Kuldeep Isharwal**, roll number **230103038**.
Repository: **https://github.com/KuldeepIsharwal/kd-cloud-parser** (commit tagged `final`). Model release: **.../releases/tag/model-v1**
Environment (measured): Windows 11 laptop running WSL2 Ubuntu 24.04 with 16 vCPU and 11 GB RAM for WSL. Docker 29.1.3, k3d 5.7.5, k3s v1.30.6, kubectl 1.31.4, Ollama 0.5.4 (container), Python 3.12, torch 2.5.1+cpu, transformers 4.46.3. No GPU.

> **Reading guide.** Measured means the number was produced by a command whose output is saved in `results/` (file names given in brackets). Assumption marks values I did not measure: design choices, prices from the rate card, or reasoning. All scores come from the kit's `score.py`, and all load numbers from the kit's `locustfile.py` with the standard command (4 users, 2 min, through the Ingress at `localhost:8080`).

### Architecture

A single-node k3d cluster (`cluster/k3d.yaml`) runs three namespaces:

- `data`: SeaweedFS in one `weed server -s3` Pod on a 5 Gi local-path PVC, exposed as `seaweedfs-s3:8333` and on NodePort 30333, which maps to `localhost:8333` on the laptop.
- `pipeline`: an Indexed Job of 4 labelling workers.
- `app`: the parser Deployment, Service, Traefik Ingress and HPA.

The teacher (phi3.5 in Ollama) runs on the laptop, outside the cluster. A selector-less Service `teacher.pipeline` with a manual Endpoints entry points at the Docker gateway. Model weights move this way: training on the laptop, then `s3://models/models/vN/`, then the service downloads them at start-up. The same tarballs are published as GitHub Release assets. S3 credentials are generated randomly by `scripts/make_secrets.sh` into a git-ignored folder and reach Pods only through Kubernetes Secrets.

## Task 1: Object storage and parallel teacher labelling

**Implementation.** The cluster is created from `cluster/k3d.yaml`. The `datasets` and `models` buckets were created with boto3, and `commands_pool.jsonl` was uploaded to `datasets` [`task1_cluster_storage.txt`]. The prompt is in `prompts/teacher_prompt.txt`: eight intents with one example each, the three slot rules, 5 few-shot examples (none taken from the validation set), and "reply with the JSON object only". `worker/worker.py` calls `/api/generate` with `temperature 0`, `seed 42`, `num_ctx 1024` and `stop ["\n"]`. It writes `{"id","text","teacher_raw"}` to `datasets/teacher/shard-<i>.jsonl`, re-uploading every 25 records so that a restarted Pod resumes instead of starting over.

**Sharding (≤ ½ page).** The pool is read as a list of lines, and worker *i* takes every line *n* with *n* mod `SHARD_COUNT` = *i*. The Job uses `completionMode: Indexed` with `completions = parallelism = 4`. Kubernetes gives each Pod a unique completion index, passed to the container as `SHARD_INDEX` through the downward API (`batch.kubernetes.io/job-completion-index`), with `SHARD_COUNT=4`. Modulo sharding needs no coordination and no shared state, gives equal shard sizes (750 each), and spreads duplicate commands and topics evenly. Each worker writes only its own object, so there are no write conflicts. Because the shards are disjoint and cover every residue, every id lands in exactly one shard. Checked: 3,000 pool ids, 3,000 ids across the shards, 0 missing, 0 repeated, and each shard equals its residue class [`task1_shard_coverage.txt`].

| Input | (a) not JSON | (b) wrong keys | (c) bad intent | (d) slot not in command | (e) duplicates | Kept |
|---|---|---|---|---|---|---|
| 3,000 | 129 | 5 | 3 | 250 | 39 | **2,574** |

Measured [`clean_counts.json`]: 129 + 5 + 3 + 250 + 39 + 2,574 = 3,000. `clean.py` checks the reasons in order (first failure counts) and removes duplicates among the valid records (smallest id kept, case and whitespace ignored). It writes `datasets/v1/train.jsonl` in id order plus `train.sha256` = `2810b6a5…580a0a`. Re-running it on the stored shards reproduces the same checksum [`task1_checksum_check.txt`: MATCH].

**Results / observations.** The 4 workers ran concurrently in `pipeline`, from 07:45 to 16:11 UTC, which is 8 h 26 min of wall time (measured). That is ~0.1 commands/s in total. A single sequential call took ~3 s. Four parallel clients sharing one CPU-bound Ollama (2 slots) mainly queue, so the parallelism shows the pattern rather than speeding anything up. Typical rejections [`task1_rejection_examples.txt`]:

- (d) invented slots, e.g. *place* = "*steve's location or office*", or *date* = "*today*" for "in the morning".
- (a) replies that started pretty-printed (`{` followed by a newline), which the newline stop cut off.
- (c) `"intent": null` for out-of-domain text.

**Limitations / improvements.** The newline stop costs ~4% of replies. Ollama's JSON mode (`format: json`) or a stop on `}` would keep them. Plain modulo sharding cannot rebalance a slow worker; a work queue would. The first run crashed WSL (out of memory with 4 × 2,048-token contexts). I raised the WSL memory limit and set `num_ctx 1024`, `OLLAMA_NUM_PARALLEL=2`.

## Task 2: Training the student (sequence-level distillation)

**Implementation.** `training/train.py` fine-tunes `EleutherAI/pythia-70m` on the **first 1,500** records of `train.jsonl`. Each example is `Command: <text>\nJSON: <teacher reply><|endoftext|>`. The reply is the teacher's JSON re-serialised compactly; the raw reply is kept unchanged in `teacher_raw`. Loss is computed only on the reply tokens. It uses AdamW with a linear schedule and 5% warm-up. `training/config.yaml` records seed 42, lr 5e-4, batch 16, 10 epochs, max length 96 and the library versions. The seed is set in code for `random`, `numpy` and `torch`, and deterministic algorithms are on. `make train` reads only that file; `make train SUBSET=50` finished in 93 s [`make_train_subset50.txt`]. v2 (`training/v2.yaml`) uses seed 7 and 12 epochs.

Each version writes a model + tokenizer + `version.json`, a deterministic tarball (sorted entries, mtime 0), and `vN.model.sha256`. The first line of that file is the tarball hash; the other lines hash every file. Files are uploaded to `s3://models/models/v1|v2/` [`task2_models_bucket.txt`] and the tarballs plus checksums are published as Release assets. v1 tarball SHA-256 = `e8cc8849…046f39`, v2 = `5e29f9ae…0f3e08`.

**Training** (measured, 16 vCPU): v1 took 2,069 s, with loss 0.889 → 0.541 → 0.263 → … → 0.0078 at epoch 10 (`training/loss.csv`). v2 took 2,202 s, ending at loss 0.0017 (`training/loss_v2.csv`).

| Model (validation.jsonl, 100 commands) | JSON valid | Intent | Date | Time | Place | Full frame |
|---|---|---|---|---|---|---|
| Teacher phi3.5 (same prompt/settings) [`teacher_validation.json`] | 96 | 84 | 72 | 73 | 79 | 41 |
| Untrained Pythia-70M [`untrained_validation.json`] | 0 | 0 | 0 | 0 | 0 | 0 |
| **Student v1 (deployed)** [`student_v1_validation.json`] | **100** | 90 | 78 | 77 | 78 | **38** |
| Student v2 [`student_v2_validation.json`] | 99 | 85 | 77 | 80 | 78 | 43 |
| Student v1 / v2, plain greedy decoding [`*_greedy_validation.json`] | 100 / 98 | 90 / 85 | 78 / 77 | 77 / 80 | 78 / 78 | 38 / 43 |

**What changed.** The untrained model continues the text with YAML-like or arbitrary output and never produces the 4-key object (0%). After distillation the student almost always emits the schema (100% valid) and reaches 38/41 = 93% of the teacher's full-frame accuracy. Its intent accuracy is higher than the teacher's (90 vs 84). That is because the cleaning step removed the teacher's malformed and invented answers, so the student learned from a filtered teacher.

One student failure mattered in deployment. For "do i need to take the umbrella out today", v1 produced the non-existent intent `weather_remove`. `score.py` still counts that as valid JSON, but the service schema rejects it, so the command failed (HTTP 500) on every load-test pass in an early run: 10 of 1,028 requests. I therefore added **schema-constrained decoding** to `training/common.py`, shared by the service and the evaluator. It forces the opening `{"intent": "`, picks the intent among the eight allowed values by the model's own log-likelihood (one batched forward pass), then generates the slots greedily. Offline scores are unchanged, and v1 now passes the service schema on 100/100 validation commands (greedy: 99/100) [`schema_check.txt`]. v2 still has one degenerate repeating output (V057).

### Sequence-level vs logit-based distillation

**(a) Sequence-level KD** (Kim & Rush, 2016) trains the student with ordinary cross-entropy on the teacher's *generated output sequence*, here the JSON string produced with greedy decoding. The teacher's single best output becomes the hard target, so the student imitates the teacher's mode rather than the original labels. **(b) Logit-based (word-level) KD** trains the student to match the teacher's full *probability distribution over the vocabulary* at every position, typically by minimising KL divergence between temperature-softened teacher and student logits. This passes on "dark knowledge": which alternatives the teacher considered likely. **(c) Why it is not possible here.** phi3.5 and Pythia-70M use different tokenizers and vocabularies (32k Llama-style vs ~50k GPT-NeoX BPE), so their per-token distributions are not over the same symbols and cannot be compared position by position. On top of that, Ollama returns only text: the

teacher's logits are not exposed. The handout also fixes the setting to stored teacher replies, with the teacher not needed after Task 1. **(d) Limitation of what I did.** The student learns only the teacher's single greedy answer, including its systematic errors. The teacher scores just 41% full-frame, and the cleaning step can only remove replies that are malformed or invent text; it cannot remove plausible but wrong ones. For example, the teacher writes "five in the morning" where the gold label is "five". So the student's ceiling is close to the teacher's. I also trained on only 1,500 examples with no held-out early stopping. The near-zero training loss (0.008) together with flat validation accuracy suggests memorisation.

## Task 3: The inference service in a container

**Implementation.** `service/app.py` is a FastAPI app on port 8000.

- `POST /parse` returns only the 4-key object, with headers `X-Model-Version` = `MODEL_VERSION` and `X-Served-By` = Pod name (downward API).
- `GET /healthz` always returns 200.
- `GET /readyz` returns 503 until a background thread has downloaded every object under `MODEL_URI` from `S3_ENDPOINT` and loaded the model, then 200.
- `MAX_NEW_TOKENS` bounds generation.
- Every output is checked against `schema.json` (keys, intent enum, string/null slots). On failure the service returns `500` `{"error":"invalid_model_output"}`; it never returns broken JSON with a 200.
- `FAULT_INJECT=1` replaces the output with `not json` before the check.
- Credentials come only from `AWS_ACCESS_KEY_ID/SECRET` environment variables injected from the `s3-credentials` Secret.

The image is `python:3.12.8-slim` plus the CPU-only torch wheel. All 40 Python packages, direct and transitive, are pinned to exact versions in `service/requirements.txt`. It contains no weights and builds with one command (`docker build -f service/Dockerfile .`, or `make image`).

| **Check (measured)** | **Result [file]** |
|---|---|
| Image size / weights in image | 1.34 GB on disk (279 MB compressed content); 0 `*.safetensors`/`*.bin` files [`task3_image.txt`] |
| Start-up | `/readyz` 503 while downloading; log: "model download done: 285,286,534 bytes in 1.75 s", then 200 [`task3_api.txt`] |
| `docker run --cpus 1 --memory 1g`, 50 sequential requests | 50 × HTTP 200; mean 480 ms, p50 476 ms, max 602 ms [`task3_50_requests.txt`] |
| Peak memory (docker stats, sampled during the 50 requests) | **567.6 MiB** of the 1,024 MiB limit; OOMKilled=false, 0 restarts [`task3_peak_memory.txt`] |
| `FAULT_INJECT=1` | `HTTP/1.1 500`, body `{"error":"invalid_model_output"}` with both headers [`task3_fault_inject.txt`] |

**Limitations / improvements.** The image is 1.34 GB mostly because of torch. ONNX Runtime or int8 quantisation would roughly halve both the image and the RAM, and fp16/bf16 weights would halve the 282 MB download. Generation is serialised by a lock (1 thread per Pod), so per-Pod throughput is ~1.8 req/s.

## Task 4: Kubernetes deployment, scaling and reliability

**Implementation** (`k8s/app/parser.yaml`):

- Namespace `app`; Deployment with 2 replicas, `requests cpu 500m / mem 512Mi`, `limits cpu 1 / mem 1Gi`.
- Startup and liveness probes on `/healthz`; readiness probe on `/readyz` every 2 s.
- RollingUpdate with `maxSurge 1, maxUnavailable 0`, and a `preStop sleep 8` so Traefik removes a Pod before it stops.
- ClusterIP Service; Traefik Ingress `/parse` → Service.
- HPA: min 1, max 2, CPU 70%, 60 s scale-down window.

| **Experiment** | **Result (measured) [file in results/]** |
|---|---|
| Ingress | POST `localhost:8080/parse` → 200 with `X-Model-Version: v1`, `X-Served-By: parser-…` [task4_basics.txt] |
| No ready endpoints while loading | After scaling 0→1: endpoints `ready=[] notReady=[10.42.0.x]` for ~7 s, until the log shows "model loaded and ready" [task4_readiness.txt] |

| | |
|---|---|
| HPA 1→2→1 | Load started 06:35:41. CPU reached 70% at 06:36:01 and 200% at 06:36:11, giving **2 Pods**. Load ended 06:39:41, CPU fell to 0% by 06:40:18, and the HPA returned to **1 Pod** at 06:41:09. Events: "New size: 2 … above target", "New size: 1 … All metrics below target". 805 requests, 0 failed [task4_hpa_*] |
| Rolling update v1→v2, then `rollout undo` | Under 5 min of load: 951 requests, **3 failed = 0.32%** (≤ 0.5%). 721 answered by v1 and 228 by v2. All 3 failures were HTTP 500 from v2's degenerate output, not dropped connections. After undo, 5/5 responses show `X-Model-Version: v1` [task4_rolling_*] |
| Recovery | Pod deleted at 06:52:31 under load: ready 1/2, back to **2/2 at 06:52:44 (13 s)**; 637 requests, **0 failed** [task4_recovery_*] |
| Cold start (delete all Pods, 3 runs) | Creation→Ready of the slowest Pod: 10, 8, 9 s, **median 9 s**. Logs of all 6 new Pods show `model download start: s3://models/models/v1` … `download done: 285286534 bytes` [task4_coldstart.txt] |
| Load balancing (exactly 2 Ready) | Standard Locust run through the Ingress: replica A **217 (50.2%)**, replica B **215 (49.8%)**, 0 failed [task4_lb_*] |

**Limitations.** The cluster is **one k3s node on one laptop**. Every replica, SeaweedFS and the control plane share one machine, one kernel and one disk, so this shows Pod-level self-healing but **not fault tolerance across machines**: losing the laptop or WSL loses everything. I saw this happen. Heavy disk I/O made k3s's SQLite datastore slow (8 s queries), and the server container exited once and did not restart by itself (a WSL cgroup error), so I had to run `docker start`. Other limitations:

- "Start with two replicas" conflicts with HPA min 1: when idle, the HPA scales to 1 within ~1 min.
- metrics-server needs ~60 s after a Pod starts before the HPA has data (`<unknown>` in events).
- CPU utilisation is measured against the 500m request, so one busy Pod reads 200%.
- SeaweedFS is a single replica on a local-path volume, a single point of failure.

**Improvements.** A multi-node cluster (3 servers + agents) with Pod anti-affinity, a PodDisruptionBudget, replicated object storage, a startupProbe tuned to the cold start, and higher HPA headroom (max > 2).

## Task 5: Subnetting and network access control

| Subnet (need) | Network | Prefix | Usable host range | Broadcast | Usable |
|---|---|---|---|---|---|
| app (1,000) | 10.20.0.0 | /22 | 10.20.0.1 – 10.20.3.254 | 10.20.3.255 | 1,022 |
| public (250) | 10.20.4.0 | /24 | 10.20.4.1 – 10.20.4.254 | 10.20.4.255 | 254 |
| data (100) | 10.20.5.0 | /25 | 10.20.5.1 – 10.20.5.126 | 10.20.5.127 | 126 |
| management (20) | 10.20.5.128 | /27 | 10.20.5.129 – 10.20.5.158 | 10.20.5.159 | 30 |

Working (`network/subnets.md`): the smallest prefix whose $2^{32-p}-2 \geq$ need. 1,000 needs $2^{10}$ (/22); 250 needs /24 ($2^7-2 = 126$ is too small); 100 needs /25; 20 needs /27. Placing them largest first, back to back from 10.20.0.0, each block start is naturally aligned. I checked this with Python's `ipaddress` (`network/check_subnets.py`).

**NACL** (`network/nacl.json`, public subnet):

- **Inbound:** 100 deny all from 198.51.100.0/24; 110 allow tcp 80 from 0.0.0.0/0; 120 allow tcp 443 from 0.0.0.0/0; 130 allow tcp 22 from 10.20.5.128/27; 140 allow tcp 1024–65535 from 10.20.0.0/22 (replies from the app tier); default deny.
- **Outbound:** 100 allow tcp 8000 to 10.20.0.0/22; 110 allow tcp 1024–65535 to 0.0.0.0/0 (replies to clients); default deny.

`nacl_sim.py --check`: 6/6 on `packets_sample.jsonl`, and 15/15 on 15 edge-case packets I wrote (`network/packets_extra.jsonl`: abusive range on 80 and UDP, SSH from the management, app and outside ranges, ephemeral replies, the wrong protocol).

**Why replies need their own rules.** A NACL is **stateless**: it does not remember that it allowed a request, so each reply packet is evaluated again on its own. A client's request to port 443 comes back as a reply to the client's ephemeral port (1024–65535), and the LB's request to an app server on 8000 comes back to the LB's ephemeral port. Without outbound rule 110 and inbound rule 140, those replies hit the default deny and every connection breaks, unlike a stateful security group. **Why order matters.** Rules are evaluated from the lowest number up and the first match wins. 198.51.100.9 → tcp 443 matches both deny 100 and allow 120. Because the deny has the lower number, it wins. If it were numbered after 110/120 (say 200), the web allow would match first and the abusive range would get through on 80/443. **Limitation.** Rule 110 (outbound ephemeral to anywhere) also allows outbound traffic to high ports such as

5432 on the data tier. A stateless NACL cannot tell a reply from a new connection, so the security groups must do that part.

## Task 6: Reproducibility, testing and CI/CD

**Bring-up.** `git clone … && cd kd-cloud-parser && make up`, three commands. `make up` runs these steps:

- creates a venv;
- creates the cluster and deploys SeaweedFS;
- generates the Secrets;
- creates the buckets and uploads the dataset;
- downloads v1/v2 from the Release, verifies their SHA-256 and uploads them to `s3://models`;
- builds and imports the image;
- applies the manifests and waits until `/parse` answers through the Ingress.

`make down` runs `k3d cluster delete`. The result of a fresh-clone test is in `results/task6_fresh_clone_make_up.txt`.

**CI** (`.github/workflows/ci.yml`) runs in this order: **lint** (ruff) → **unit tests** (16 pytest tests covering sharding, every `clean.py` rejection reason, determinism, the service API and headers, fault injection, invalid outputs, readiness, subnets and NACL) → **model evaluation** (`gh release download model-v1`, verify the tarball's SHA-256 against `v1.model.sha256`, predict on `validation.jsonl`, run `score.py`) → **quality gate** (`score.py --min-validity 95`). A separate **gitleaks** job scans the full history. A `workflow_dispatch` input `model=untrained` swaps in the base Pythia-70M.

- Passing run (trained model): **RUN_PASS_URL**.
- **Run failing at the quality gate** (untrained model, 0% validity): **RUN_FAIL_URL**.
- gitleaks result: **GITLEAKS_RESULT**.

`AI_USE.md` lists the AI tool used and how each output was checked.

**Limitations / improvements.** CI evaluates on 100 validation examples only, and the gate checks only validity. Adding a full-frame threshold and building and smoke-testing the container in CI would catch more regressions.

## Task 7: Performance, cost and review

**Standard load test** (measured, 2 Ready replicas, through the Ingress) [`loadtest_stats.csv`, `task7_locust_stdout.txt`, `task7_top_pods.txt`]:

- 432 requests, **0 failures (error rate 0.00%)**.
- **p50 1,100 ms, p95 1,300 ms** (max 2,353 ms); throughput **3.63 req/s**.
- `kubectl top` during the load: each Pod at **1,002–1,008m CPU** (pinned at its 1-core limit) and **503–765 MiB** memory (362/355 MiB when idle).
- Cold start: median **9 s** (Task 4).
- Requests split 217/217 between the two replicas [`loadtest_headers.json`].

**Cost** (rates from rate_card.json, INR; each replica billed for its limits) [`task7_cost_slo.txt`]:

- Per replica: 1 vCPU × 3.50 + 1 GiB × 0.45 = 3.95 INR/h.
- 2 replicas: 7.90 INR/h.
- Requests per hour: 3.6276 req/s × 3,600 = 13,059.2.
- **Cost per 1,000 requests = 7.90 / 13,059.2 × 1,000 = 0.605 INR.**

(Assumption: the replicas are fully busy. At lower load the same hourly cost is spread over fewer requests.)

**SLO.** 99% of `POST /parse` requests return HTTP 200 within 30 s, over the window of one standard load-test run (2 min). Error budget = (1 − 0.99) × 432 = 4.32, so 4 failed requests are allowed. **Used: 0 (0% of the budget).** In the rolling-update run (951 requests) the same SLO would allow 9 failures and 3 were used (33%).

| Component | Evidence (measured) | Limitation found | Improvement proposed |
|---|---|---|---|
| Data and storage | 2,574/3,000 replies kept (85.8%); 250 invented slots; model download 285 MB in 1.1–1.75 s | Single SeaweedFS replica on one local volume; 4% of replies lost to the newline stop | Replicated object storage; Ollama JSON mode; store the clean dataset versioned with its checksum (done for v1) |
| Model quality | Full frame: student 38%, teacher 41%; JSON validity 100%; intent 90% | Inherits teacher errors; train loss 0.008 suggests | Better teacher prompt/self-consistency, a held-out set for early stopping, more data, |

| | | | |
|---|---|---|---|
| | | memorisation; v2 produced 1/100 degenerate outputs | constrained JSON decoding of the slots too |
| Latency and throughput | p50 1,100 ms, p95 1,300 ms, 3.63 req/s with 2 Pods; 476 ms p50 for one sequential client | One request at a time per Pod; under load ~60% of latency is queueing | Batching, ONNX/int8, KV-cache reuse of the shared prompt prefix, more replicas |
| Resource utilisation | ~1,005m CPU per Pod under load (100% of limit); memory 503–765 MiB of 1 GiB; peak 568 MiB in Docker at 1 CPU | CPU-bound; memory grows under concurrency and leaves ~25% headroom | Quantise to int8 (smaller RAM), CPU limit > 1 with more torch threads |
| Reliability and recovery | Recovery 13 s with 0 failures; rolling update 0.32% failed; cold start median 9 s | Single node: no machine-level fault tolerance; k3s container once stopped and needed a manual restart | Multi-node cluster, PodDisruptionBudget, anti-affinity, a restart watchdog |
| Security and cost | 0.605 INR / 1,000 requests at 7.90 INR/h; Secrets only via the K8s Secret; gitleaks result in CI | Secrets are base64, not encrypted at rest; plain HTTP; replicas billed even when idle | Sealed Secrets/KMS, TLS on the Ingress, scale-to-zero or a lower min replica count off-peak |

### References

Kim & Rush, Sequence-Level Knowledge Distillation, EMNLP 2016. Hinton et al., Distilling the Knowledge in a Neural Network, 2015. Chen et al., Distilling Knowledge Learned in BERT for Text Generation, ACL 2020. Bastianelli et al., SLURP, EMNLP 2020 (data, CC BY 4.0). EleutherAI Pythia-70M model card. Microsoft Phi-3.5-mini-instruct (via Ollama). Documentation for k3d, k3s, Kubernetes (HPA, probes, Secrets, rolling updates), SeaweedFS, FastAPI, Docker, GitHub Actions, gitleaks, AWS VPC Network ACLs, and the Google SRE book (SLOs).